APB Timer IP Verification Report

Block-Level Verification

1 DUT Overview

The verified design consists of an `apb_timer` wrapper containing two timer instances by default. Each timer is accessed through the APB interface and contains three software-visible registers:

Register	Offset	Purpose
TIMER	`0x0`	Current timer value / programmable timer count
TIMER_CTRL	`0x4`	Timer enable and prescaler configuration
CMP	`0x8`	Compare value for compare interrupt generation

The control register uses bit 0 as the timer enable and bits [5:3] as the prescaler. Each timer provides two interrupt outputs: compare interrupt and overflow interrupt.

For the default configuration `TIMER_CNT = 2`, address bit `PADDR[4]` is used by the wrapper to select the timer instance for the current APB transaction. Thus, `PADDR[4] = 0` selects Timer 0, while `PADDR[4] = 1` selects Timer 1. The wrapper asserts the internal `PSEL` only for the selected timer and multiplexes its `PRDATA`, `PREADY`, and `PSLVERR` response back to the APB master.

The resulting register map is:

Timer	Register	Address
Timer 0	TIMER	`0x000`
Timer 0	TIMER_CTRL	`0x004`
Timer 0	CMP	`0x008`
Timer 1	TIMER	`0x010`
Timer 1	TIMER_CTRL	`0x014`
Timer 1	CMP	`0x018`

The APB address selects which timer is being accessed; it does not determine whether that timer is running. Each timer is independently enabled through bit 0 of its own `TIMER_CTRL` register. Therefore, after configuration, Timer 0 and Timer 1 can operate simultaneously and independently.

2 Verification Methodology

A self-checking environment was developed around the original RTL. The environment contains a reusable APB transaction path, passive monitoring, a reference-model scoreboard, functional coverage, and protocol/design assertions.

The essential verification flow is:

```
Sequence -> Sequencer -> APB Driver/BFM -> DUT
                                |
DUT/APB signals -> Monitor -------+
                                |
                        Scoreboard / Coverage
```

The APB driver acts as the active bus-functional layer. It converts transaction-level read/write requests into correct APB setup and access phases. The monitor independently observes completed

APB transfers, while the scoreboard performs functional checking.

This separation allows the APB stimulus mechanism to be reused for other APB peripherals in the SoC, while timer-specific behavior remains in the timer sequences, reference model, assertions, and coverage.

3 Key Verification Scenarios

Seven verification scenarios were executed within the main test. They were selected to cover both normal operation and important timer corner cases.

No.	Scenario	Purpose
1	Register Test	Verify APB read/write behavior of timer, control, and compare registers.
2	Prescaler Test	Exercise prescaler values 0–7 and verify timer counting behavior.
3	Compare IRQ Test	Program compare values and verify compare interrupt generation for both timers.
4	Overflow Test	Drive the timer close to `0xFFFFFFFF` and verify overflow interrupt behavior.
5	CMP Zero Test	Verify the special case where CMP is zero and compare interrupt must not be falsely generated.
6	Independence Test	Program the two timer instances independently and verify that activity in one does not corrupt the other.
7	Random Test	Apply 5000 constrained-random APB transactions across valid timer registers and configurations.

The directed scenarios guarantee that critical features are intentionally exercised, while constrained-random traffic explores combinations and ordering that are less practical to enumerate manually.

4 Reference Model and Scoreboard

The scoreboard is the primary functional checker of the environment. It maintains an independent model of the important state of both timer instances, including:

- timer count,
- control register value,
- compare register value,
- internal prescaler/cycle-counter behavior, and
- expected compare and overflow interrupts.

At each relevant clock cycle, the reference model calculates the expected next state using the intended RTL behavior. Actual DUT outputs are then compared against these expected values. APB read data, interrupt behavior, `PREADY`, and `PSLVERR` are checked automatically.

An important aspect of the model is that it reproduces the DUT's update priority. In particular, an APB write can override the automatic timer update/reset in the same next-state calculation. This prevents the scoreboard from using an oversimplified timer model and allows corner-cycle behavior to be checked accurately.

5 Assertions

SystemVerilog assertions were added to detect protocol and interface violations independently of the scoreboard. The important properties include:

- `PREADY` remains asserted as required by this zero-wait-state slave.
- `PSLVERR` remains deasserted as implemented by the DUT.
- An APB setup phase is followed by the access phase.
- Address and control information remain stable across the transfer.
- Write data remains stable during an APB write transfer.
- `PENABLE` is not entered without the preceding setup phase.
- Interrupt outputs remain inactive while the design is in reset.

Assertions provide a second checking mechanism: the scoreboard checks functional results, while assertions check important temporal/protocol rules.

6 Functional Coverage Strategy

Functional coverage was included to measure whether important stimulus combinations were exercised. Coverage points were defined for:

- timer instance selection,
- timer/control/compare register accesses,
- APB read versus write operation,
- timer enable state,
- prescaler values 0–7,
- compare-value categories including CMP=0,
- individual interrupt outputs, and
- relevant crosses between timer, register, operation, enable, and prescaler selections.

Code coverage was also enabled during simulation, including block, expression, toggle, and functional coverage instrumentation. Coverage is used as a completeness metric and to identify scenarios that require additional targeted stimulus; it is not replaced by a high transaction or scoreboard-check count.

7 Simulation Results

The complete regression executed successfully using the `apb_timer_test`. All seven verification scenarios reported PASS.

Result	Observed Value
Verification scenarios passed	7 / 7
Scoreboard checks	19,464
Scoreboard errors	0
Timer 0 compare IRQ observations	3
Timer 0 overflow IRQ observations	1
Timer 1 compare IRQ observations	3
Timer 1 overflow IRQ observations	1
errors	0
fatal errors	0

The regression therefore produced no scoreboard mismatch, error, or fatal error. Both compare and overflow interrupt paths were observed for both timer instances.

It is important to distinguish the 19,464 scoreboard checks from the number of tests. The regression contains one main test executing seven verification sequences. The check count represents the large number of expected-versus-actual comparisons performed by the self-checking environment during those scenarios.

8 Verification Confidence and SoC Integration Status

The verification results provide strong evidence that the APB Timer IP behaves correctly for the functionality exercised by the current verification plan. In particular, the environment has checked register programming, timer counting, prescaling, compare and overflow interrupts, CMP-zero behavior, multi-timer independence, APB behavior, and randomized register traffic without detecting a functional mismatch.

Based on the completed regression:

- all seven planned verification scenarios passed,
- 19,464 automated scoreboard checks completed with zero errors,
- both interrupt mechanisms were exercised on both timer instances,
- no error or fatal condition was reported, and
- protocol/design assertions were active during the regression.

Therefore, **no functional error was detected within the verified scope**, and the IP is a strong candidate for SoC-level integration testing. At SoC level, verification can focus on integration aspects such as address decoding, interconnect/bridge access, reset/clock connectivity, and end-to-end interrupt routing rather than repeating all block-level timer corner cases.

9 Conclusion

A reusable, self-checking environment was successfully used to verify the APB Timer IP. Verification combined directed feature tests, constrained-random stimulus, an independent cycle-accurate reference model, assertions, and coverage collection. The regression completed with **7/7 scenarios passing, 19,464 checks, and zero scoreboard/ errors**.

The results demonstrate that the timer is functioning correctly across the verified operating conditions and is suitable to proceed to SoC integration verification, subject to final coverage review and the project's normal sign-off criteria.